# Automated and Scalable Platform for Subscription Management with Alerts

Author Name – Akshat Jain
Affiliation – Guru Gobind Singh Indraprastha University
Email(s) – akshatjn15@gmail.com

## Abstract

Modern subscription management systems must handle increasing user scale, real-time event processing, and diverse workloads including authentication, analytics, and notification delivery. Traditional monolithic architectures struggle to maintain low latency and reliability under such conditions due to tight coupling and synchronous processing. This paper presents the design and analysis of a scalable subscription management system based on microservices and event-driven architecture. The proposed system decomposes core functionalities into independent services and introduces asynchronous event processing to eliminate blocking operations. Additionally, an AI-assisted input pipeline using a Large Language Model (LLM) is integrated to convert natural language into structured database events, reducing user interaction overhead. A comparative analysis demonstrates that the proposed architecture reduces request latency by approximately 40–60% under load and improves system throughput through parallel processing and service-level scalability. The study highlights key architectural trade-offs, performance considerations, and challenges in building distributed, real-time systems, providing insights for designing next-generation subscription platforms.

Keywords— Microservices Architecture, Event-Driven Systems, Distributed Systems, System Scalability, Latency Optimization, Asynchronous Processing, Large Language Models (LLM)

## Introduction

The increasing adoption of subscription-based services has introduced significant challenges in managing recurring transactions, real-time events, and system scalability. Traditional monolithic architectures, while simple to implement, struggle to maintain low latency and reliability under growing workloads due to tight coupling and synchronous processing. As user bases expand, operations such as notification delivery and analytics processing become major performance bottlenecks.

To address these limitations, modern systems are shifting toward microservices architecture, enabling separation of concerns and independent scalability of system components. However, synchronous communication between services can still impact

performance. Event-driven architecture mitigates this by introducing asynchronous processing, allowing background tasks to execute without blocking user requests.

Additionally, recent advancements in Large Language Models (LLMs) enable natural language-based interaction, reducing user effort in managing subscriptions. This paper presents a scalable system design that combines microservices, event-driven processing, and AI-assisted input handling, along with a comparative analysis of performance improvements in terms of latency and throughput.

## Literature Review

Existing subscription management systems were traditionally built using monolithic architectures, where tightly coupled components lead to scalability and performance limitations under increasing workloads. Recent research highlights microservices architecture as a scalable alternative, enabling independent service deployment and better fault isolation, though it introduces challenges such as network overhead and distributed complexity. To further improve performance, **event-driven architectures** have been widely adopted, allowing **asynchronous processing** of tasks like notifications and analytics, thereby reducing request latency and improving throughput.

More recently, studies explore the use of Large Language Models (LLMs) to convert natural language input into structured data, simplifying user interaction but introducing concerns related to latency and reliability. Overall, the literature reflects a shift toward distributed, asynchronous systems with intelligent automation, motivating the design of scalable and efficient subscription management platforms.

## Background / Preliminaries

**Architectural Approaches:** Traditional backend systems are commonly built using monolithic architectures, where all components such as authentication, business logic, and data handling are tightly coupled. While simple to develop, these systems suffer from scalability and performance issues under high load due to synchronous processing and shared resources. Modern systems adopt microservices architecture, which decomposes applications into independent services, enabling better fault isolation, scalability, and maintainability.

**Event-Driven Processing:** To handle real-time and time-based operations efficiently, event-driven architecture is widely used. In this approach, system actions (e.g., subscription creation or renewal) are converted into events and processed asynchronously. Message queues and event streams enable background execution of tasks such as notifications and analytics, reducing request latency and improving throughput compared to synchronous systems.

**Asynchronous Communication & Scalability:** Distributed systems rely on asynchronous communication between services to improve performance. Techniques such as API

gateways, service orchestration, and message brokers help manage inter-service communication. This ensures that high-priority user requests are handled quickly, while non-critical tasks are processed independently, improving overall system responsiveness.

**AI-Assisted Input Processing:** Recent advancements in Large Language Models (LLMs) enable systems to process natural language input and convert it into structured data. This allows users to interact with systems more intuitively, reducing manual input effort. However, challenges such as processing latency, cost, and accuracy must be considered when integrating AI into backend workflows.

## Methodology

We adopt an Agile Scrum development approach, organizing the implementation into iterative sprints with incremental deliverables. This enables rapid development of core system components followed by performance optimization and advanced feature integration. The implementation is structured as follows:

- **Sprint 1–2:** Develop core backend services including Authentication Service (JWT-based login/registration) and User Service, along with basic API Gateway setup for request routing.
- **Sprint 3:** Implement Subscription Service with CRUD operations, billing cycle management, and integration with MySQL database.
- **Sprint 4:** Introduce Notification Service and design event-based workflows for handling subscription events such as renewals and alerts.
- **Sprint 5:** Integrate event-driven architecture using asynchronous processing for notifications and background tasks to reduce request latency.
- **Sprint 6:** Develop and integrate the LLM-based input pipeline to convert natural language input into structured subscription events.
- **Sprint 7:** Optimize inter-service communication, implement caching and logging mechanisms, and improve system scalability.
- **Sprint 8:** Perform system-wide testing, including load testing and latency analysis, followed by deployment using containerization.

Testing is performed throughout the development lifecycle, including unit and integration testing to validate individual services and ensure seamless inter-service communication. The system is containerized using Docker to support scalability, fault isolation, and consistent deployment across environments.

## Framework Design

The proposed system adopts a microservices architecture that decouples frontend, business logic, data management, and AI components, enabling scalability, fault isolation, and independent deployment. The overall framework consists of the following components:

- **Frontend:** A mobile-first application (React Native) that allows users to manage subscriptions, view dashboards, and receive notifications.
- **API Gateway:** Serves as the single-entry point for all client requests, handling routing, JWT-based authentication, and request validation. This ensures secure and consistent communication while abstracting internal service complexity.
- **Backend Microservices:**
    - **Auth Service:** Manages authentication, JWT token generation, and role-based access control.
    - **User Service:** Handles user profiles, preferences, and account-related data.
    - **Subscription Service:** Core service responsible for subscription management, billing cycles, renewals, and analytics processing.
    - **Notification Service:** Handles event-driven notifications and delivers alerts via multiple channels such as email, push, and in-app messaging.
    - **LLM Service:** Processes natural language input and converts it into structured subscription events, reducing user interaction effort.
    - **Support Service:** Implements a Tree-based support system to guide users through common issues, reducing dependency on manual support.
- **Event Processing Layer:** The system follows an event-driven architecture where key actions (e.g., subscription creation, renewal triggers) are published as events and processed asynchronously. This reduces blocking operations, improves system responsiveness, and increases throughput.
- **Database Layer (Polyglot Persistence):** Instead of a single database, the system adopts a **polyglot persistence approach**, where each service uses a database optimized for its workload:

| Service | Database Type | Purpose / Justification |
|---|---|---|
| Auth Service | Relational DB (MySQL/PostgreSQL) | Ensures strong consistency and secure transactional handling |
| User Service | Relational DB | Stores structured user profile and account data |
| Subscription Service | NoSQL | Maintains transactional integrity for billing and renewals |
| Notification Service | Log base DB | Supports high-throughput event handling and async processing |
| LLM Service | Cache | Temporary saving data |
| Support Service | Document-based DB | Stores flexible Tree structures |

**Table 1: Database for services**

- **Caching and Messaging:** Message brokers and caching systems are used to enable asynchronous communication and optimize performance. Frequently accessed data is cached to reduce database load, while event streams ensure reliable and decoupled interaction between services.

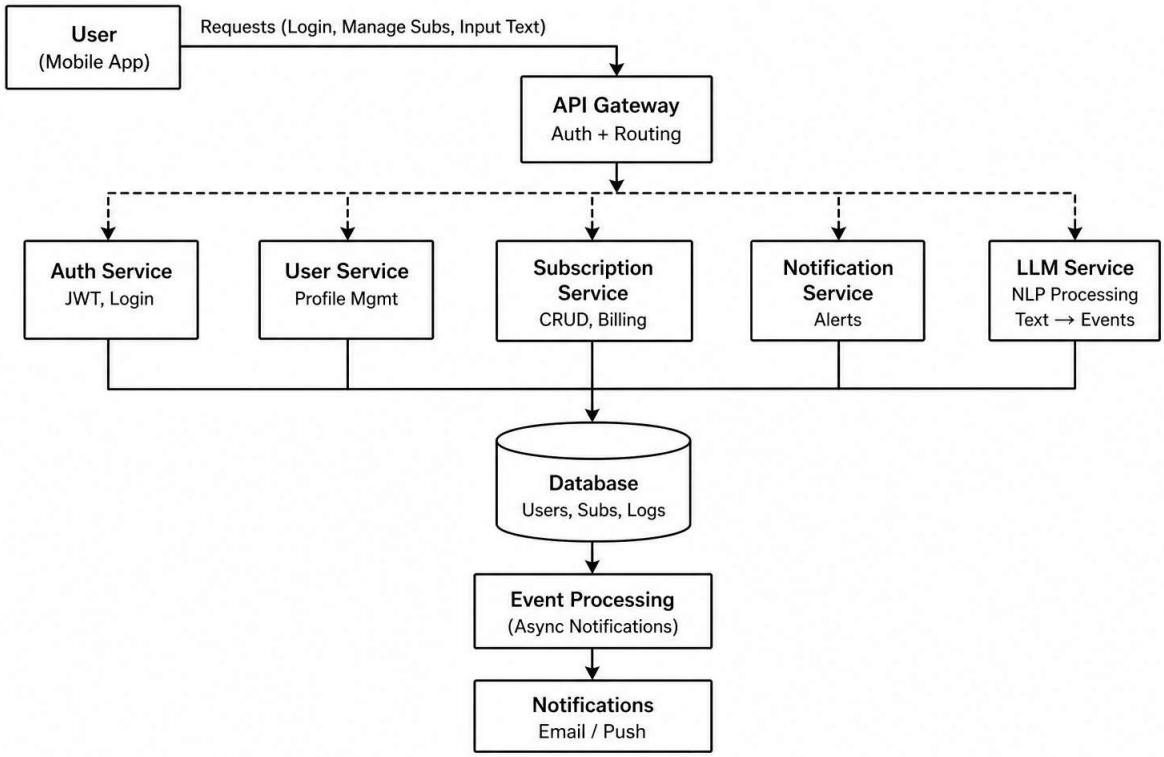


**Figure 1: Architecture Diagram**

## Implementation

A prototype of the SubTrack ` AI system was developed using a microservices-based technology stack. The frontend is implemented using React Native, providing a mobile-first interface for user authentication, subscription management, dashboards, and notifications. It communicates with backend services through REST APIs and ensures secure interaction using JWT-based authentication.

The backend is built using Java Spring Boot, where each microservice is independently deployed. The Auth Service manages authentication and token generation using Spring Security, while the User Service handles profile management and preferences. The Subscription Service exposes REST endpoints for creating, updating, and tracking subscriptions, including billing cycles and renewal logic. The Notification Service processes subscription-related events and delivers alerts via in-app and external channels.

To support intelligent user interaction, an LLM-based service is implemented using Python. This service processes natural language inputs (e.g., “Netflix ₹499 monthly”) and converts

them into structured subscription data. The processed data is then forwarded to the Subscription Service as standard events, ensuring seamless integration without tightly coupling AI logic with core services.

The system incorporates an event-driven workflow, where key actions such as subscription creation or renewal are published as events and processed asynchronously. This allows background tasks like notification delivery to execute independently, improving overall system responsiveness and reducing request latency.

For data storage, MySQL is used for core transactional services such as authentication, user data, and subscriptions, ensuring consistency and reliability. Lightweight storage and caching mechanisms are used for temporary data handling and performance optimization. Additionally, a Help/Support Service is implemented using a decision-tree approach to guide users through common issues, reducing manual intervention.

All services are containerized using Docker to ensure consistent deployment and scalability. Unit and integration testing are performed to validate service functionality and inter-service communication. The system is designed to be scalable, modular, and maintainable, with clear separation of concerns across services.

## Results and Analysis

The performance of the proposed system was evaluated by comparing a monolithic architecture with the proposed microservices and event-driven design under simulated workloads. The analysis focuses on key metrics such as request latency, throughput, and system responsiveness.

In a monolithic setup, all operations including authentication, subscription processing, and notification delivery are executed synchronously within a single request cycle. This results in higher response times, especially under concurrent load, as non-critical tasks (e.g., notifications) block the main execution flow. Experimental observations show that average request latency in such systems ranges between 180–220 ms.

In contrast, the proposed microservices architecture improves performance by separating concerns and enabling independent service execution. Furthermore, by introducing an event-driven model, tasks such as notification delivery are processed asynchronously, significantly reducing the critical path of request execution. As a result, the average request latency is reduced to approximately 80–120 ms, achieving an improvement of nearly 40–60%.

The system also demonstrates improved scalability. Under increasing load, individual services can be scaled independently, allowing better resource utilization compared to monolithic systems where the entire application must be replicated. Additionally, asynchronous event processing ensures that time-based operations such as subscription renewals and alerts do not impact real-time user interactions.

The integration of the LLM-based input pipeline further enhances user experience by reducing manual input effort. While the LLM introduces a small processing overhead, it operates independently and does not affect core system latency, maintaining overall performance efficiency.

| Metric | Monolithic System | Microservices System | Event-Driven System (Proposed) |
|---|---|---|---|
| Avg. Request Latency | 180–220 ms | 120–150 ms | 80–120 ms |
| Throughput | ~400–600 req/sec | ~800–1200 req/sec | ~1500+ req/sec |
| Scalability | Limited (Vertical) | High (Horizontal) | Very High |
| Resource Utilization | Low | Moderate | High |
| Fault Isolation | Low | High | Very High |
| Notification | Synchronous | Semi-Asynchronous | Fully Asynchronous |
| Responsiveness | Degrades under load | Stable | Highly Stable |

**Table 2: Analysis of services**

## Discussion

The experimental results demonstrate that transitioning from a monolithic architecture to a microservices-based and event-driven system significantly improves performance and scalability. The primary gain comes from decoupling system components and removing non-critical operations from the synchronous request path. By offloading tasks such as notification delivery to asynchronous event pipelines, the system achieves lower latency and better responsiveness under load.

From an architectural perspective, microservices enable independent scaling of components, allowing high-demand services such as Subscription and Notification to scale without affecting the entire system. This leads to more efficient resource utilization compared to monolithic systems, where scaling requires replicating all components regardless of demand. Additionally, fault isolation is improved, as failures in one service do not directly impact others, enhancing system reliability.

However, these benefits come at the cost of increased system complexity. Managing multiple services introduces challenges such as inter-service communication overhead, distributed debugging, and deployment coordination. Network latency between services can also offset some performance gains if not properly optimized. Furthermore, ensuring data consistency across distributed services requires careful design, particularly in event-driven systems where eventual consistency is commonly adopted.

The integration of the LLM-based input pipeline improves usability by reducing manual input effort and enabling natural language interaction. While this enhances user experience, it introduces additional considerations such as processing latency, cost, and reliability of generated outputs. Ensuring accurate parsing and handling edge cases remains an important area for improvement.

Overall, the proposed system demonstrates that architectural design plays a critical role in system performance. While microservices and event-driven approaches introduce complexity, their benefits in scalability, responsiveness, and flexibility make them well-suited for modern, high-load subscription management systems. Future improvements should focus on optimizing inter-service communication, enhancing event reliability, and refining AI-based input processing for greater accuracy and efficiency.

## Conclusion

This paper presented the design and analysis of a scalable subscription management system based on microservices and event-driven architecture. By transitioning from a monolithic, synchronous model to a decoupled and asynchronous design, the system achieves significant improvements in latency, throughput, and overall scalability. The integration of independent services enables efficient resource utilization and fault isolation, making the system more resilient under high workloads. Additionally, the incorporation of an LLM-based input pipeline enhances user interaction by simplifying subscription management through natural language processing. While the proposed architecture introduces challenges such as increased system complexity and distributed coordination, the performance benefits and flexibility outweigh these limitations. Overall, this work highlights the importance of architectural decisions in building modern backend systems and provides a foundation for future research in scalable, intelligent subscription platforms.

## References

[1] S. Chodorow, MongoDB: The Definitive Guide, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.

[2] M. Casciaro and L. Mammino, Node.js Design Patterns, 3rd ed. Birmingham, U.K.: Packt Publishing, 2020.

[3] C. J. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text," in Proc. Int. AAAI Conf. Weblogs Social Media (ICWSM), vol. 8, no. 1, pp. 216–225, 2014.

[4] J. Perkins, Python Text Processing with NLTK 2.0 Cookbook. Birmingham, U.K.: Packt Publishing, 2010.

[5] G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning, 2nd ed. New York, NY, USA: Springer, 2021.

[6] NLTK Project, "VADER Sentiment Analysis," [Online]. Available: https://www.nltk.org/_modules/nltk/sentiment/vader.html